ACKNOWLEDGEMENT

We would like to express our profound gratitude to **Dr. Jyoti Pareek (HOD)** of Department of Computer Science, Gujarat University for their contributions to the completion of our project **Colorization of Black and White Images**. We would like to express our special thanks to our mentor **Shri. Eric Shah** and guide **Shri Ankan Majumdar** for their time and efforts they provided throughout the year. Your useful advice and suggestions were helpful to us during the project's completion. Without their constant guidance, support, help and encouragement the present study would not have been possible. In this aspect, We all eternally grateful to you. I would like to acknowledge that this project was completed entirely by me and my team and not by someone else.

INDEX

Abstract

Black and White Image colorization is the process of adding color to grayscale or black and white images. It can be done manually by artists or automatically using computer algorithms. Deep learning-based approaches have gained popularity in recent years for automatically colorizing black and white images. These methods use neural networks, specifically convolutional neural networks (CNNs) and generative adversarial networks (GANs), to learn the mapping between grayscale and color images from a large dataset of color images.

Deep learning is an existing function of AI that works similarly like a human brain. Example, it processes the data and creates patterns for the use in decision making. Machine learning is the superset of deep learning. Deep learning has networks which are capable of learning independently form of data that is unlabelled.

Deep learning technology demands high resources. It requires high-performance and more powerful GPUs, large amounts of space to store the data that is used to teach the models, so on. Unlike the traditional machine learning, this technology takes more time to be trained. Though deep learning has all the challenges, it is still being used because it has been discovering new improved methods of unstructured big data analytics day-by-day.

Introduction

Image colorization is the process of taking an input grayscale (black and white) image and then producing an output colorized image that represents the semantic colors and tones of the input.

Color images contain more information than gray scale image. And it is more useful to extract information from color image and it is visually appealing to viewers.

Color image consists of three-dimensional information whereas gray scale image consists of luminance, and it is one dimensional.

Convolutional Neural Network

A **convolutional Neural Network (CNN)** is a type of deep learning neural network architecture commonly used in computer vision.

Computer Vision is a field of artificial intelligence that enables a computer to understand and interpret the image or visual data. Neural networks are used in various datasets like images, audio, and text.

Different types of neural networks are used for different purposes, for image classification we will use convolution neural networks.

Neural Network Layers

In a regular neural network, there are three types of layers:

Input layers: It is the layer in which we give input to our model. The number of neurons in this layer is equal to the total number of features in our data or number of pixels.

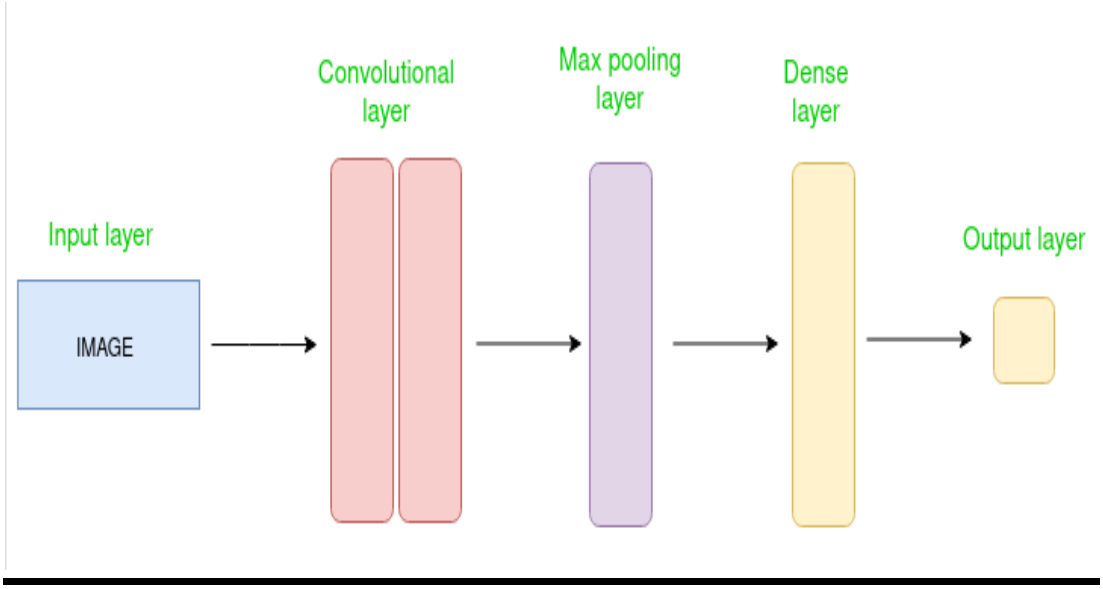
Hidden layer: The input from the input layer is then feed into the hidden layer. There can be many hidden layers depending upon our model and data size. Each hidden layer can have different numbers of neurons which are generally greater than the number of features. The output from each layer is computed by matrix multiplication of output of the previous layer with learnable weights of that layer and then by the addition of learnable biases.

Output layer: The output from the hidden layer is then fed into a logistic function like sigmoid which converts the output of each class into the probability score of each class.

CNN Architecture

A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data.

Convolutional Neural Network consists of multiple layers like the input layer, convolutional layer, pooling layer, and fully connected layers.



The Convolutional layer applies filters to the input image to extract features, the Pooling layer down samples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

Types of layers in CNN

Input layers: It is the layer in which we give input to our model. In CNN, generally, the input will be an image or a sequence of images.

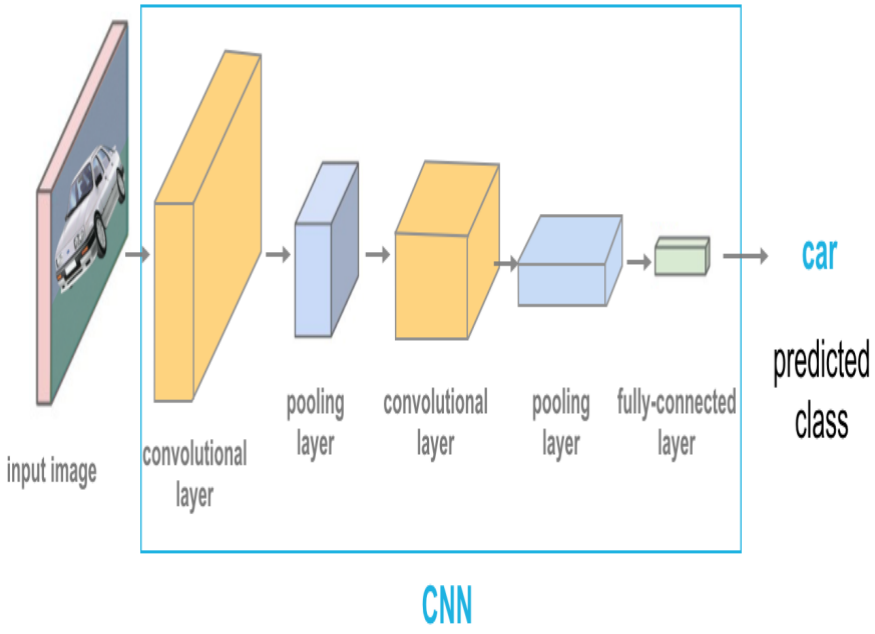
Convolutional layers: This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices that slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch.

Pooling layer: The main function is to reduce the size of volume which makes the computation fast reduces memory and prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**.

Flattening: The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.

Fully connected layers: It takes the input from the previous layer and computes the final classification or regression task.

Output layer: The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid which converts the output of each class into the probability score of each class.



Literature Review

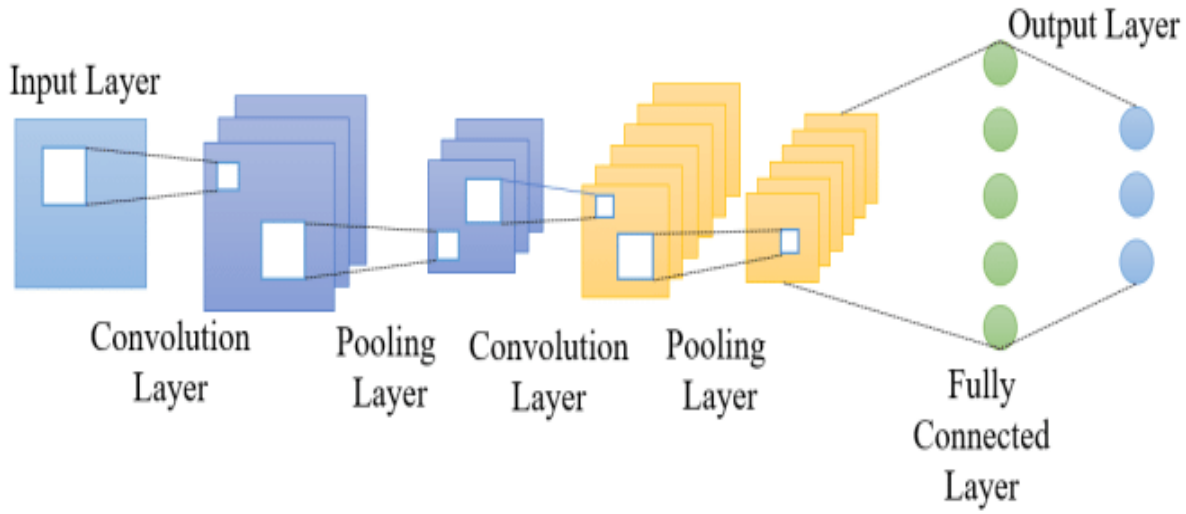
Richard Zhang has proposed an optimized solution by using huge data-set and single feed-forward pass in CNN. Their focus lies on training part. They used human subjects to test the results and were able to fool 32% of them. It can have various number of neurons. The various attempts used various architectures. In some research, generally number of neurons is same as the dimension of the feature descriptor extracted from each pixel coordinates in a Gray-scale image.

Global features: Most of the methods utilize the global features to form an image filter, and then use this filter to select similar images from a large image set automatically. However, some models produced unnatural colorization result due to global similarity but semantic difference.

Data Set size: Parametric and Non- Parametric models use different sizes of data sets for training the CNN. The Parametric model use very large data set to train the CNN and produce more accurate result while the non-Parametric models rely more on the input hints and reference image and use smaller data set for training the CNN.

Semantic Information: Semantic information has a significant and unavoidable role in deep image colorization. For effective colorization of images, the system must have information of the semantic composition of the image and its localization. For instance, leaves on a tree may be coloured some kind of green in spring, but they should be coloured brown for a scene set in autumn. VGG-16 CNN model was used by majority of approaches to extract semantic information about the image before applying colorization techniques.

Feature Extraction: Features of the image are obtained through by integrating pre-trained neural networks to extract information about objects, shapes and use this context to assign color values to the objects. Some approaches used to use Inception ResNet V2 classifier or TensorFlow to serve this purpose.



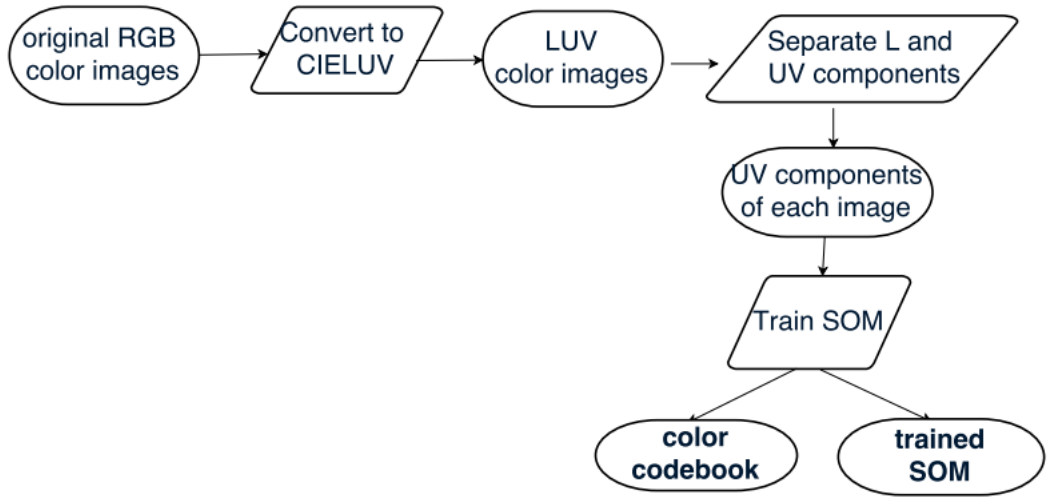
Methodology

Black and white image colorization is the process of adding color to grayscale or black and white images to make them appear as if they were originally captured in color. There are several methodologies and techniques used to accomplish this task, and here it is described a commonly used approach known as deep learning-based colorization.

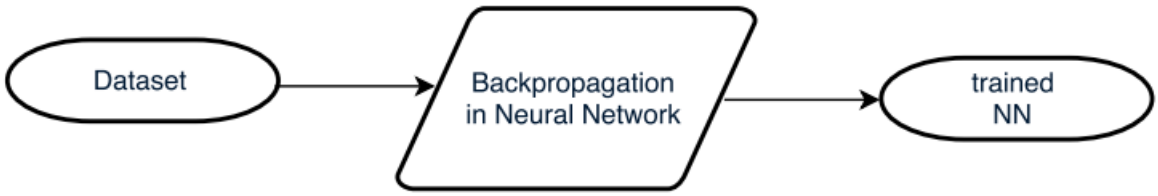
Data Collection: The first step is to gather a large dataset of coloured images along with their corresponding grayscale versions. This dataset is used to train a deep learning model to learn the mapping between grayscale and color images.

Network Architecture: The next step involves designing a deep neural network architecture that can effectively learn and generate color information from grayscale inputs. Commonly used architectures for colorization tasks include convolutional neural networks (CNNs) and generative adversarial networks (GANs).

Preprocessing: Before training the network, the grayscale images and their corresponding color images need to be pre-processed. This may involve resizing the images, normalizing pixel values, and applying any necessary transformations to improve the quality of the training data.



Training: The network is then trained using the pre-processed dataset. During training, the network learns to predict color information from grayscale images by minimizing a loss function that quantifies the difference between the predicted colours and the ground truth colours in the training dataset. This process typically involves optimizing the network's weights using techniques such as backpropagation and gradient descent.



Inference: Once the network is trained, it can be used to colorize new grayscale images. Given a grayscale input image, the network predicts the corresponding color information using the learned mapping. This can be done by passing the grayscale image through the trained network and obtaining the colorized output.

Postprocessing: The colorized output may undergo postprocessing steps to enhance its visual quality. This can involve techniques such as color correction, contrast adjustment, and smoothing to make the colorized image appear more realistic and visually pleasing.

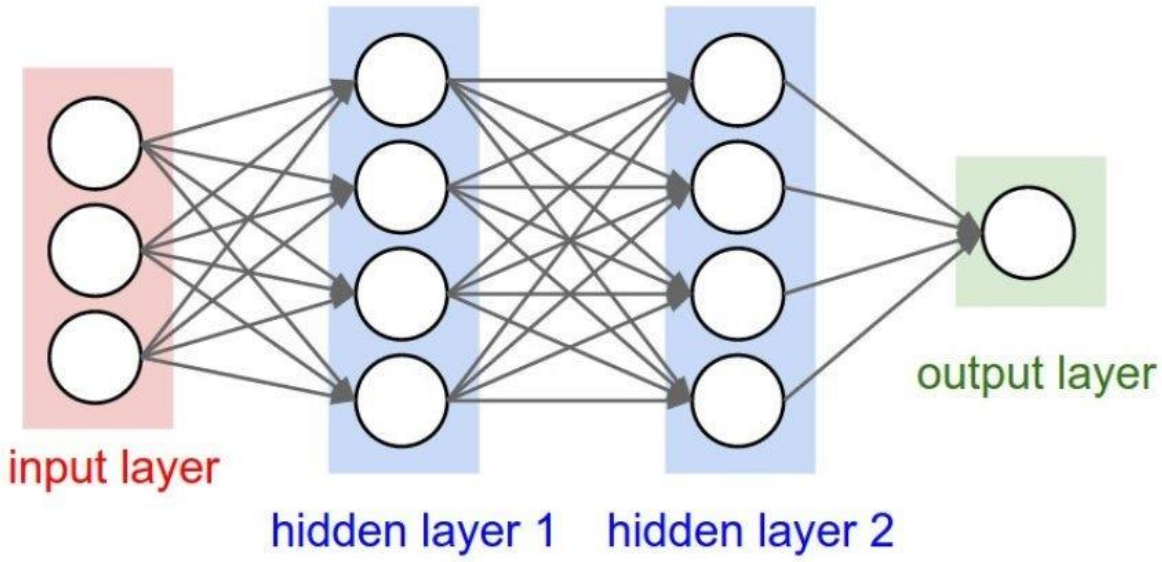
Deep Learning

Deep learning is a subset of machine learning, which is essentially a neural network with three or more layers. These neural networks attempt to simulate the behavior of the human brain—albeit far from matching its ability—allowing it to “learn” from large amounts of data. While a neural network with a single layer can still make approximate predictions, additional hidden layers can help to optimize and refine for accuracy.

How deep learning works

Deep learning neural networks, or artificial neural networks, attempts to mimic the human brain through a combination of data inputs, weights, and bias. These elements work together to accurately recognize, classify, and describe objects within the data.

Deep neural networks consist of multiple layers of interconnected nodes, each building upon the previous layer to refine and optimize the prediction or categorization. This progression of computations through the network is called forward propagation. The input and output layers of a deep neural network are called *visible* layers. The input layer is where the deep learning model ingests the data for processing, and the output layer is where the final prediction or classification is made.

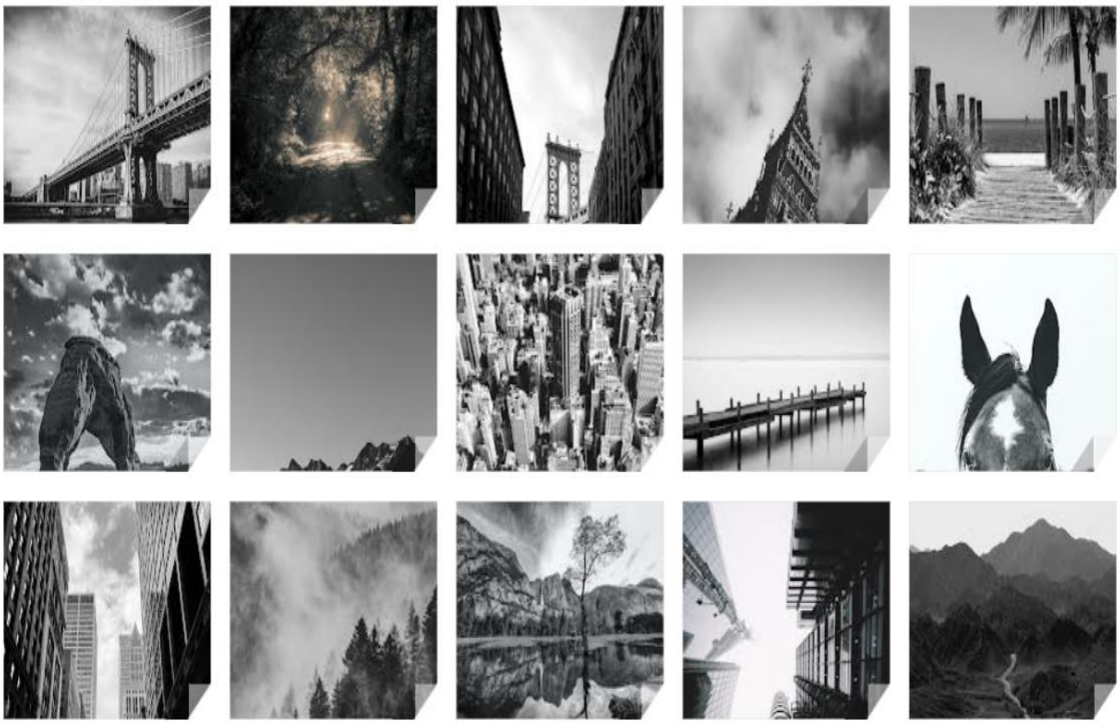


Another process called backpropagation uses algorithms, like gradient descent, to calculate errors in predictions and then adjusts the weights and biases of the function by moving backwards through the layers in an effort to train the model. Together, forward propagation and backpropagation allow a neural network to make predictions and correct for any errors accordingly. Over time, the algorithm becomes gradually more accurate.

Dataset

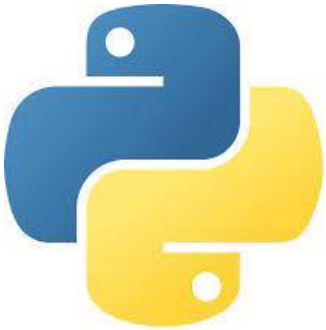
The dataset created for the project is collection of black and white images which are to be converted to Colour Images through the model.

Collection of Black and White images:



Tools, Technologies and Principles Used

Python



Python is a high-level, general-purpose, and very popular programming language. Python programming language is being used in web development, Machine Learning applications, along with all cutting-edge technology in Software Industry.

Python programs generally are smaller than other programming languages like Java. Programmers have to type relatively less and the indentation requirement of the language, makes them readable all the time.

Libraries Used

NumPy

NumPy stands for numeric python which is a python package for the computation and processing of the multidimensional and single dimensional array elements.

Travis Oliphant created NumPy package in 2005 by injecting the features of the ancestor module Numeric into another module Numarray.

It is an extension module of Python which is mostly written in C. It provides various functions which are capable of performing the numeric computations with a high speed.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

```
[1 2 3 4 5]<class 'numpy.ndarray'>
```

Sklearn

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modelling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python. This library, which is largely written in Python, is built upon NumPy, SciPy and Matplotlib.

```
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
```

```
le=LabelEncoder()
```

```
le
```

```
LabelEncoder()
```

```
bp_train,bp_test=train_test_split(df,test_size=0.2)
```

```
bp_train_x=bp_train.iloc[: ,1:-1]
bp_test_x=bp_test.iloc[: ,1:-1]
```

Pickle

Pickle in Python is primarily used in serializing and deserializing a Python object structure. In other words, it's the process of converting a Python object into a byte stream to store it in a file/database, maintain program state across sessions, or transport data over the network.

```
import pickle
```

```
pickle.dump(dtc, open("health.pickle", 'wb'))
```

```
loaded_model = pickle.load(open("/content/health.pickle", 'rb'))
result = loaded_model.score(bp_test_x, bp_test_y)
print(result)
```

```
0.835
```

Tkinter

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit. Import the Tkinter module.

PIL

The Python Pillow library is a fork of an older library called PIL. PIL stands for Python Imaging Library, and it's the original library that enabled Python to deal with images.

OpenCV

OpenCV is a huge open-source library for computer vision, machine learning, and image processing. OpenCV supports a wide variety of programming languages like Python, C++, Java, etc. It can process images and videos to identify objects, faces, or even the handwriting of a human.

Keras

Keras is an effective high-level neural network Application Programming Interface (API) written in Python. This open-source neural network library is designed to provide fast experimentation with deep neural networks, and it can run on top of CNTK, TensorFlow, and Theano.

Keras focuses on being modular, user-friendly, and extensible. It doesn't handle low-level computations; instead, it hands them off to another library called the Backend.

TensorFlow

TensorFlow is an end-to-end open-source deep learning framework.

It is known for documentation and training support, scalable production and deployment options, multiple abstraction levels, and support for different platforms, such as Android.

TensorFlow is a symbolic math library used for neural networks and is best suited for dataflow programming across a range of tasks. It offers multiple abstraction levels for building and training models.

Implementation Details

PROCESS

The whole process for Image Colorization could be as:

1. Convert all training images in Lab color space from RGB color space using Open CV.

2. For the input to the network, use L channel and then train the network to predict the ab channels that is the combination of chrominance and luminance which is done through the Caffe model.

3. Caffe model consists of Image Dataset and prototxt file where the 8 layers using the 4 basic layers of CNN which are convolution layer, pooling, flattening and fully connection.

4. Merge the input L channel and predicted ab channels with the help of fully connected layer.

5. Transform the Lab image back to RGB by using Open CV.

Code

```
import tkinter as tk
from tkinter import *
from tkinter import filedialog
from PIL import Image, ImageTk
import os
import numpy as np
import cv2 as cv
import os.path


numpy_file = np.load('./model/pts_in_hull.npy')
Caffe_net = cv.dnn.readNetFromCaffe("./model/colorization_deploy_v2.prototxt.txt", "./model/colorization_release_v2.caffemodel")
numpy_file = numpy_file.transpose().reshape(2, 313, 1, 1)

class Window(Frame):
    def __init__(self, master=None):
        Frame.__init__(self, master)

        self.master = master
        self.pos = []
        self.master.title("Black and White Image Colorization")
        self.pack(fill=BOTH, expand=1)

        menu = Menu(self.master)
        self.master.config(menu=menu)
```

```
        file = Menu(menu)
        file.add_command(label="Upload Image", command=self.uploadImage)
        file.add_command(label="Color Image", command=self.color)
        menu.add_cascade(label="File", menu=file)

        self.canvas = tk.Canvas(self)
        self.canvas.pack(fill=tk.BOTH, expand=True)
        self.image = None
        self.image2 = None


    def uploadImage(self):
        filename = filedialog.askopenfilename(initialdir=os.getcwd())
        if not filename:
            return
        load = Image.open(filename)

        load = load.resize((480, 360), Image.ANTIALIAS)

        if self.image is None:
            w, h = load.size
            width, height = root.winfo_width(), root.winfo_height()
            self.render = ImageTk.PhotoImage(load)
            self.image = self.canvas.create_image((w / 2, h / 2), image=self.render)

        else:
            self.canvas.delete(self.image3)
            w, h = load.size
            width, height = root.winfo_screenmmwidth(), root.winfo_screenheight()
```

```
        frame = cv.imread(filename)

        Caffe_net.getLayer(Caffe_net.getLayerId('class8_ab')).blobs = [numpy_file.astype(np.float32)]
        Caffe_net.getLayer(Caffe_net.getLayerId('conv8_313_rh')).blobs = [np.full([1, 313], 2.606, np.float32)]

        input_width = 224
        input_height = 224

        rgb_img = (frame[:,:,[2, 1, 0]] * 1.0 / 255).astype(np.float32)
        lab_img = cv.cvtColor(rgb_img, cv.COLOR_RGB2Lab)
        l_channel = lab_img[:,:,0]

        l_channel_resize = cv.resize(l_channel, (input_width, input_height))
        l_channel_resize -= 50

        Caffe_net.setInput(cv.dnn.blobFromImage(l_channel_resize))
        ab_channel = Caffe_net.forward()[0,:,:,:].transpose((1,2,0))

        (original_height,original_width) = rgb_img.shape[:2]
        ab_channel_us = cv.resize(ab_channel, (original_width, original_height))
        lab_output = np.concatenate((l_channel[:,:,np.newaxis],ab_channel_us),axis=2)
        bgr_output = np.clip(cv.cvtColor(lab_output, cv.COLOR_Lab2BGR), 0, 1)


        cv.imwrite("./result.png", (bgr_output*255).astype(np.uint8))
```

```
    def color(self):

        load = Image.open("./result.png")
        load = load.resize((480, 360), Image.ANTIALIAS)

        if self.image is None:
            w, h = load.size
            self.render = ImageTk.PhotoImage(load)
            self.image = self.canvas.create_image((w / 2, h/2), image=self.render)
            root.geometry("%dx%d" % (w, h))
        else:
            w, h = load.size
            width, height = root.winfo_screenmmwidth(), root.winfo_screenheight()

            self.render3 = ImageTk.PhotoImage(load)
            self.image3 = self.canvas.create_image((w / 2, h / 2), image=self.render3)
            self.canvas.move(self.image3, 500, 0)


root = tk.Tk()
root.geometry("%dx%d" % (980, 600))

app = Window(root)
app.pack(fill=tk.BOTH, expand=1)
root.mainloop()
```

Results

Evaluation

Advantages of Convolutional Neural Networks

1. Good at detecting patterns and features in images, videos, and audio signals.
2. Robust to translation, rotation, and scaling invariance.
3. End-to-end training, no need for manual feature extraction.
4. Can handle large amounts of data and achieve high accuracy.

Disadvantages of Convolutional Neural Networks

1. Computationally expensive to train and require a lot of memory.
2. Can be prone to overfitting if not enough data or proper regularization is used.
3. Requires large amounts of labeled data.
4. Interpretability is limited, it’s hard to understand what the network has learned.

Colorization Procedure Using A Generative Adversarial Network (GAN)

- Images presented using GAN Model are not presenting information properly as CNN model does.

- GAN doesn't provide much clear output as some parts remained gray after colorization process.

- It is not presenting accurate output as it is not using CNN caffe model which is training the images using different layers and combines the output to provide the colorized image.

Conclusion

We have presented a method of fully automatic colorization of unique grayscale images combining state of-the-art CNN techniques.

Using color representation and the right loss function, we have represented that the method is capable of producing a plausible and vibrant colorization of certain parts of images that has properties which may be applied to video sequences also.

Our model does very well with the animals like cats and dogs because the dataset we chose i.e., ImageNet consists large amount of pictures of these animals. Even the outdoor scenes turnout very good with our model. The model also captures notion of sunset and paints it orange.

The model produces plausible images even with the sketches. Finally, the model also works well with the black and white videos in most of the cases.

References

https://www.geeksforgeeks.org/deep-learning-with-python-opencv/

https://learnopencv.com/

https://pyimagesearch.com/2017/08/21/deep-learning-with-opencv/

https://learnopencv.com/deep-learning-and-computer-vision-demos/

https://machinelearningmastery.com/introduction-python-deep-learning-library-tensorflow/